

02477 – Bayesian Machine Learning: Lecture 9

Michael Riis Andersen

Technical University of Denmark,
DTU Compute, Department of Applied Math and Computer Science

Outline

- 1 Markov chain Monte Carlo
- 2 Advanced sampling methods
 - Hamiltonian dynamics
 - Hamiltonian Monte Carlo
 - The MALA and ULA algorithms
- 3 Convergence diagnostics

Markov chain Monte Carlo

Bayesian inference and probabilistic modelling

■ Bayesian supervised learning in general:

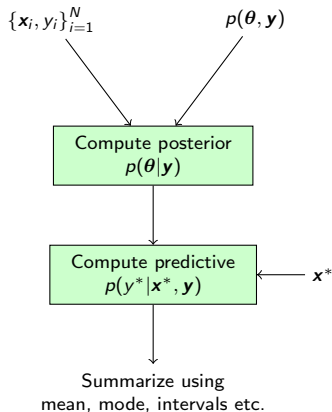
1. Joint model of data $\mathbf{y}$ and parameters θ .
2. Summarize knowledge of θ given data $\mathbf{y}$.
3. Compute posterior predictive distribution.

■ Goal: separate modelling from inference:

1. Build models reflecting domain knowledge.
2. Push “inference button” and get results.

■ Inference methods:

1. Laplace approximations.
2. Markov chain Monte Carlo.
3. Variational approximations.



Monte Carlo: Posterior inference using samples

- Many posterior summaries can be phrased as expectations $\mathbb{E}_p[f(\boldsymbol{\theta})]$ for some function f

- We can compute expectations wrt. p using the *Monte Carlo estimator*

$$\bar{f} = \mathbb{E}_p[f(\boldsymbol{\theta})] = \int f(\boldsymbol{\theta})p(\boldsymbol{\theta})d\boldsymbol{\theta} \approx \frac{1}{S} \sum_{i=1}^S f(\boldsymbol{\theta}^i) \equiv \hat{f},$$

where $\boldsymbol{\theta}^i \sim p(\boldsymbol{\theta})$ are *independent samples* for $i = 1, \dots, S$

- We showed last time that ...

1. the Monte Carlo estimator $\hat{f}$ is *unbiased*
2. the variance of $\hat{f}$ decreases with $1/S$ when the samples are *i.i.d.*

A zoo of sampling-based methods

■ Simple sampling methods

1. Rejection sampling
2. Ancestral sampling
3. Importance sampling
4. Transformation methods
5. Inverse transform sampling
6. ...

■ MCMC methods

1. Metropolis-Hastings
2. Gibbs Sampling
3. Slice sampling
4. Hamiltonian Monte Carlo
5. ...

MCMC using the Metropolis-Hastings algorithm

- We can use the MH to generate samples from a distribution of interest $p_t(\theta)$.

The Metropolis-Hastings algorithm

- Start from some initial value θ^0 (e.g., a sample from the prior).
- Repeat for $k = 1$ to K :
 1. Given last value θ^{k-1} , generate *candidate sample* using proposal distribution

$$\theta^* \sim q(\theta^* | \theta^{k-1}).$$

2. Compute *acceptance probability* A_k as follows

$$A_k = \min \left(1, \frac{p_t(\theta^*)q(\theta^{k-1} | \theta^*)}{p_t(\theta^{k-1})q(\theta^* | \theta^{k-1})} \right).$$

3. *Accept* θ^* with probability A_k

$$\theta^k = \begin{cases} \theta^* & \text{if } u_k < A_k \\ \theta^{k-1} & \text{otherwise} \end{cases}, \quad \text{for } u_k \sim \mathcal{U}(0, 1)$$

- What do we need in order to implement MH for a given model?
- High flexibility, but the price we pay is correlation in generated samples

Markov chain Monte Carlo theory I

- Metropolis-Hastings defines a chain of samples $\theta^0, \theta^1, \theta^2, \dots$ with a *Markov property*

$$p(\theta^{k+1} | \theta^1, \theta^2, \dots, \theta^k) = p(\theta^{k+1} | \theta^k)$$

- The *transition kernel* tells us how to iterate the chain

$$T(\theta^{k+1} | \theta^k) \equiv p(\theta^{k+1} | \theta^k)$$

- The distribution of θ^{k+1} is given by *sum rule*

$$p(\theta^{k+1}) = \int T(\theta^{k+1} | \theta^k) p(\theta^k) d\theta^k$$

- A distribution $p^*(\theta)$ is said to be *invariant* or *stationary* wrt. the Markov chain if each step does not change the distribution

$$p^*(\theta) = \int T(\theta | \theta') p^*(\theta') d\theta'$$

- We require $p^*(\theta)$ to be a limiting distribution of the chain (*independent of the initial distribution*).

$$p(\theta^k) \rightarrow p^*(\theta) \quad \text{for} \quad k \rightarrow \infty$$

Transition kernel for Metropolis-Hastings

- Recall the acceptance probability for Metropolis-Hastings

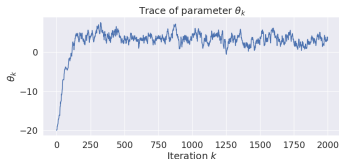
$$A(\theta^*|\theta^k) = \min \left[1, \frac{p(\theta^*)q(\theta^k|\theta^*)}{p(\theta^k)q(\theta^*|\theta^k)} \right]$$

- The transition kernel for Metropolis-Hastings

$$T(\theta'|\theta) = \begin{cases} q(\theta'|\theta)A(\theta'|\theta) & \text{if } \theta' \neq \theta \\ q(\theta|\theta)A(\theta|\theta) + \int q(\theta''|\theta) [1 - A(\theta''|\theta)] d\theta'' & \text{if } \theta' = \theta \end{cases}$$

- The big picture

1. We set θ^0 to some initial value (*initialization*)
2. We iterate $\theta^{k+1}|\theta^k \sim T(\theta^{k+1}|\theta^k)$ (*warm-up phase*)
3. Eventually the distribution of θ^k will converge to the stationary distribution p^* (*sampling phase*)



Markov chain Monte Carlo theory II

$$p^*(\theta) = \int T(\theta|\theta')p^*(\theta')d\theta'$$

- We require $p^*(\theta)$ to be a limiting distribution of the chain (*independent of the initial distribution*).

$$p(\theta^k) \rightarrow p^*(\theta) \quad \text{for} \quad k \rightarrow \infty$$

- Conditions required for a Markov chain to have a limiting distribution equal to the unique stationary distribution:

(A1) *Irreducible*: all states θ can be reached.

Counter example: $w^{k+1} = w^k + |e_k|$, where $e_k \sim \mathcal{N}(0, 1)$

(A2) *Aperiodic*: no deterministic cycles.

Counter example: $w^1 = 1, w^2 = 2, w^3 = 3, w^4 = 1, w^5 = 2, \dots$

(A3) *Positive recurrent*: the chain has positive probability of returning to any given state (+ finite expected return-time).

$$P(\theta^k \in A | \theta^0) > 0 \quad \text{for all sets } A \quad \text{where} \quad P(A) > 0$$

- A chain that satisfies (A1)–(A3) is said to be *ergodic* and ensures $p(\theta^k) \rightarrow p^*(\theta)$.

Markov chain Monte Carlo theory III

- These conditions are generally hard to check in practice.
- Simpler condition: if a chain satisfies the *detailed balance condition*, then p^* is its stationary distribution

$$T(\theta'|\theta)p^*(\theta) = T(\theta|\theta')p^*(\theta').$$

- MH with reasonable proposal distributions satisfies detailed balance (see slide 39).
- It's often not a question of convergence or not, but rather how fast we converge for a given proposal distribution.
- For example, all (non-degenerate) Gaussian proposals lead to detailed balance, but convergence time may vary drastically depending on the proposal variance.
- More theory and details: Monte Carlo Statistical Methods by Robert and Casella.



Pros and cons of Metropolis-Hastings

Pros

- Strong mathematical guarantees: If we sample long enough, the iterates θ^k will converge to the exact target distribution

$$p(\theta^k) \rightarrow p^*(\theta) \quad \text{for} \quad k \rightarrow \infty$$

- Easy to implement.
- Easy to prototype and evaluate different models.

Cons

- May have to sample “infinitely” long for difficult distributions.
- Acceptance ratio can be low.
- Slow for large datasets.
- Proposal distribution may require tuning.

Questions: True or false?

Quiz via DTU Learn:

Lecture 9: Metropolis-Hastings (12 questions)

Check you knowledge

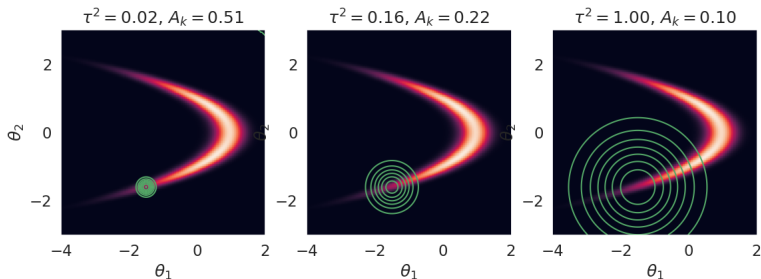
Advanced sampling methods

Designing more efficient algorithms

- We have now studied the Metropolis-Hastings algorithm with Gaussian proposals:

$$q(\theta^*|\theta^k) = \mathcal{N}(\theta^*|\theta^k, \tau^2 \mathbf{I}) \iff \theta^* = \theta^k + \tau \epsilon, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

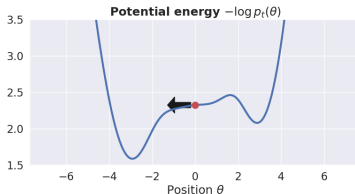
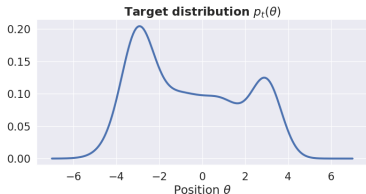
- This variant is often called the *Metropolized Random Walk* (MRW) algorithm. It is simple and easy to implement, but it *can suffer from slow convergence/mixing*



- Large proposal variance leads to proposals, where target density $p_t(\theta)$ is very low
- Structure of target distribution forces us to use small proposal variance, which leads to slow exploration. This problem becomes even *worse in high-dimensional* spaces.

Hamiltonian Monte Carlo

- The *Hamiltonian Monte Carlo* (HMC) is the state-of-the-art MCMC algorithm
 - 1987: First proposed by team working in quantum physics
 - 1996: Radford Neal that it could be applied much broader in statistical modelling
 - 2012: The Stan software package made it mainstream
 - 2014: Hoffman and Gelman proposed state-of-the-variant called "NUTS"
- HMC is a special case of the Metropolis-Hastings algorithm
- Recall: the MRW proposal distribution is *independent of target distribution* $p_t(\theta)$
$$q_{\text{MRW}}(\theta^*|\theta^k) = \mathcal{N}(\theta^*|\theta^k, \tau^2 I) \iff \theta_{\text{MRW}}^* = \theta^k + \tau \epsilon, \epsilon \sim \mathcal{N}(\mathbf{0}, I)$$
- HMC uses a clever proposal distribution that *leverages the gradient of the target distribution* $p_t(\theta)$ to guide the proposals.
- Physical analogy: particle gliding along a frictionless surface



Hamiltonian Monte Carlo Animation

Show animation for intuition

<https://chi-feng.github.io/mcmc-demo/app.html>

Advanced sampling methods: Hamiltonian dynamics

Hamiltonian Monte Carlo: Definitions and terminology

- Let $p_t(\theta)$ be the target distribution for parameters of interest $\theta \in \mathbb{R}^D$. We refer to θ as the *position* of some imaginative particle.
- Let $\nu \in \mathbb{R}^D$ denote the *momentum* of the particle and assume $p(\nu) = \mathcal{N}(\nu|\mathbf{0}, I)$
- Let $E(\theta) = -\log p_t(\theta) + \text{const}$ denote the *potential energy*.
- Let $K(\eta) = -\log p(\nu) + \text{const}$ denote the *kinetic energy*
- Define the *Hamiltonian function* $H(\theta, \nu)$ as the *total energy*

$$\begin{aligned} H(\theta, \nu) &= E(\theta) + K(\nu) + \text{const} \\ &= -\log p_t(\theta) - \log p(\nu) + \text{const} \end{aligned}$$

- Let's negate and exponentiate both sides

$$\begin{aligned} \Rightarrow e^{-H(\theta, \nu)} &= e^{\log p_t(\theta) + \log p(\nu) + \text{const}} \\ &= p_t(\theta)p(\nu)e^{\text{const}} \end{aligned}$$

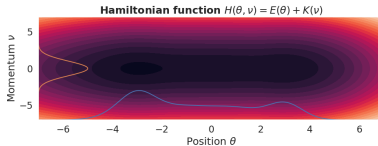
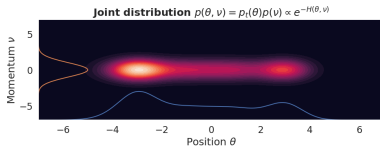
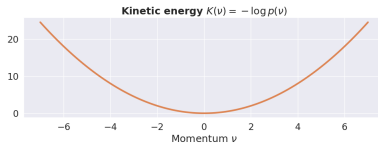
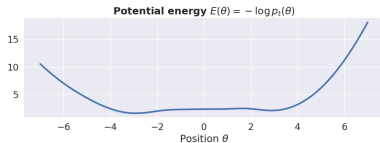
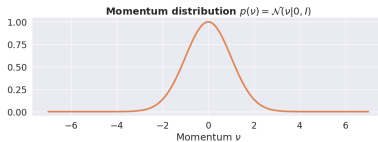
- Dividing both sides by $Z = e^{\text{const}}$ yields

$$p_t(\theta)p(\nu) = \frac{1}{Z}e^{-H(\theta, \nu)}$$

- Assume the joint distribution over *phase space* (θ, ν) is given by

$$p(\theta, \nu) = p_t(\theta)p(\nu) = \frac{1}{Z}e^{-H(\theta, \nu)}$$

Hamiltonian Monte Carlo: 1D Example

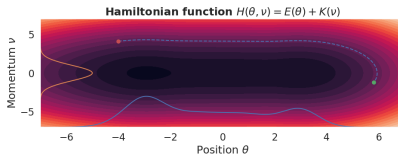


Hamilton's equations

$$E(\theta) = -\log p_t(\theta) + \text{const}$$

$$K(\nu) = -\log p(\nu) + \text{const}$$

$$H(\theta, \nu) = E(\theta) + K(\nu) + \text{const}$$



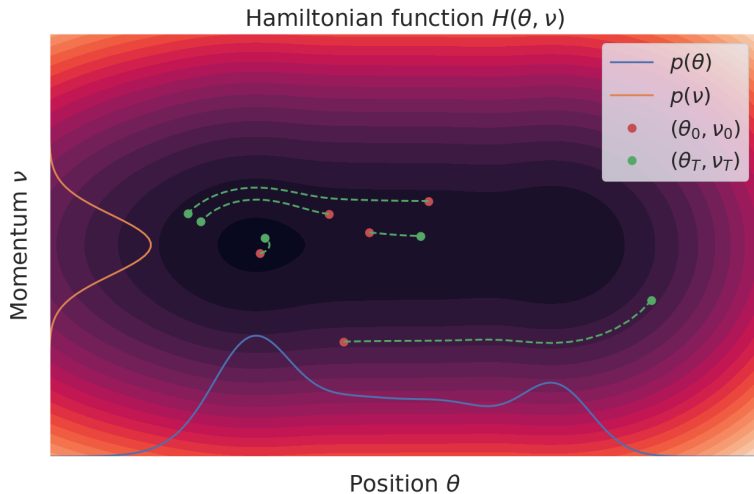
- *Hamilton's equations* describes how a particle with position θ and momentum ν moves over time

$$\begin{aligned}\frac{d\theta}{dt} &= \frac{\partial H}{\partial \nu} = \frac{\partial K}{\partial \nu} \\ \frac{d\nu}{dt} &= -\frac{\partial H}{\partial \theta} = -\frac{\partial E}{\partial \theta}\end{aligned}$$

- Given an initial state (θ_0, ν_0) , we can compute the state (θ_t, ν_t) for any given time t by *simulating/solving* Hamilton's ODEs in *discrete time*
- We can show that $\frac{dH}{dt} = 0$, i.e. the *value of the Hamiltonian H is conserved*
- We can use Hamiltonian dynamics to *propose large steps* in θ while avoiding random walk behavior!

Hamiltonian Monte Carlo: Examples

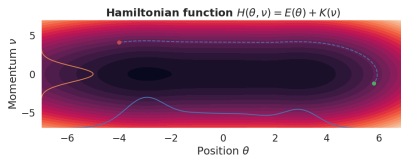
Hamiltonian Monte Carlo: Examples



Integrating Hamilton's equation: Euler's method

- We *discretize* and solve *Hamilton's equation* numerically

$$\frac{d\theta}{dt} = \frac{\partial K}{\partial \nu}, \quad \frac{d\nu}{dt} = -\frac{\partial E}{\partial \theta}$$



- Let $t \in \mathbb{R}$ denote the *continuous time index* and $k \in \mathbb{N}$ denote the *discrete time index*
- The *simplest* method is *Euler's* method with *step-size* $\eta > 0$

$$\frac{d\theta}{dt} \approx \frac{\theta(t + \eta) - \theta(t)}{\eta} \Rightarrow \theta(t + \eta) \approx \theta(t) + \eta \frac{d\theta}{dt} \Rightarrow \theta_{k+1} \approx \theta_k + \eta \frac{d\theta_k}{dt}$$

- For *Gaussian momentum*, we have $K(\nu) = \frac{1}{2}\nu^T\nu + \text{const}$, so $\nabla_{\nu}K(\nu) = \nu$.
Therefore,

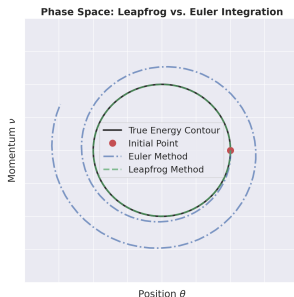
$$\begin{aligned}\theta_{k+1} &\approx \theta_k + \eta \nu_k \\ \nu_{k+1} &\approx \nu_k - \eta \nabla_{\theta} E(\theta_k)\end{aligned}$$

- Because of the *approximation error* introduced by Euler's method, we *cannot* turn this into a valid and symmetric proposal distribution

Integrating Hamilton's equation: The leapfrog method

- We use the *leapfrog method* instead because it is/has:

1. Reversible
2. Volume-preserving ($|\det J| = 1$)
3. Better approximation error



- Given a *step-size* $\eta > 0$ and a state (θ_k, ν_k) at time $t = k\eta$, we can approximate $(\theta_{k+1}, \nu_{k+1})$ using a *leapfrog integrator*:

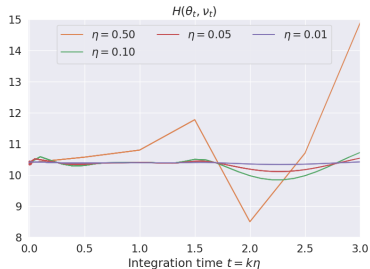
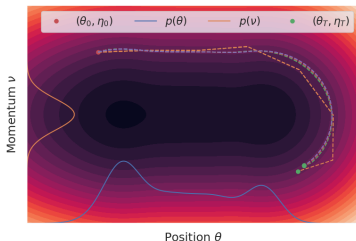
$$\nu_{k+1/2} = \nu_k - \frac{\eta}{2} \nabla_{\theta} E(\theta_k) \quad (\text{Half-step for } \nu)$$

$$\theta_{k+1} = \theta_k + \eta \nu_{k+1/2} \quad (\text{Full-step for } \theta)$$

$$\nu_{k+1} = \nu_{k+1/2} - \frac{\eta}{2} \nabla_{\theta} E(\theta_{k+1}) \quad (\text{Half-step for } \nu)$$

- The above equations correspond to *1 step* of the leapfrog integrator. We *iterate* these equations for L steps to approximate the state at time $T = L\eta$

Leapfrog integration in action I



For $k = 1, 2, \dots, L$, do:

$$\boldsymbol{\nu}_{k+1/2} = \boldsymbol{\nu}_k - \frac{\eta}{2} \nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta}_k) \quad (\text{Half-step for } \boldsymbol{\nu})$$

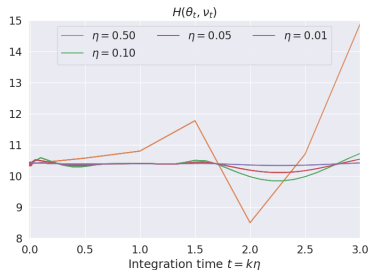
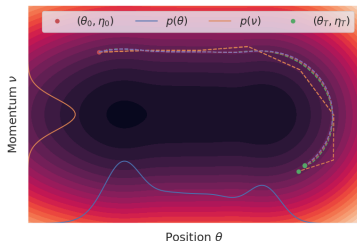
$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta \boldsymbol{\nu}_{k+1/2} \quad (\text{Full-step for } \boldsymbol{\theta})$$

$$\boldsymbol{\nu}_{k+1} = \boldsymbol{\nu}_{k+1/2} - \frac{\eta}{2} \nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta}_{k+1}) \quad (\text{Half-step for } \boldsymbol{\nu})$$

The leapfrog procedure defines a *deterministic map* $(\boldsymbol{\theta}_k, \boldsymbol{\nu}_k) \mapsto (\boldsymbol{\theta}^*, \boldsymbol{\nu}^*)$, i.e.

$$(\boldsymbol{\theta}^*, \boldsymbol{\nu}^*) = \text{leapfrog}_{\eta, L}(\boldsymbol{\theta}_k, \boldsymbol{\nu}_k)$$

Leapfrog integration in action II

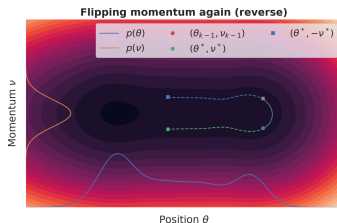


A few observations

- The total integration time is given by $T = L\eta$, where the step-size η determines the accuracy
- $\|\theta_0 - \theta_T\|$ is not a monotonic function in T
- The number of leapfrog steps L and step-size η are *hyperparameters* of the algorithm
- In practice, we have robust heuristics to automatically tune both L and η
- The Hamiltonian $H(\theta, \nu)$ is *only approximately preserved in practice*, but this is not a problem as we will see soon

Advanced sampling methods: Hamiltonian Monte Carlo

Hamiltonian dynamics as a proposal distribution



- How do we turn this into a (symmetric) *proposal distribution* $q(\theta^*, \nu^* | \theta_{k-1}, \nu_{k-1})$ distribution?
- To *propose a new candidate* (θ^*, ν^*) for the k 'th iteration in HMC, we
 1. Sample a fresh momentum vector: $\nu_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 2. Compute $(\theta^*, \nu^*) = \text{leapfrog}_{\eta, L}(\theta_{k-1}, \nu_{k-1})$
 3. Flip the momentum and return $(\theta^*, -\nu^*)$
- *Reversibility*: Running the leapfrog integrator from $(\theta^*, -\nu^*)$ and flipping the momentum returns us to $(\theta_{k-1}, \nu_{k-1})$
- In theory, we need to *flip the momentum* after integrating to ensure a *symmetric proposal distribution*, but it does not matter in practice when the kinetic energy is a zero-mean Gaussian, i.e. $p(\nu) = p(-\nu)$, so this step is often ignored in practice.

The big picture of HMC

Hamiltonian Monte Carlo algorithm

Let $\theta_k \in \mathbb{R}^D$ be the position at iteration k and let $\text{leapfrog}_{\eta,L}(\cdot, \cdot)$ be a leapfrog integrator, then each iteration of HMC proceeds as follows

For $k = 1, 2, \dots, K$, do

1. Sample *new momentum* variable $\nu_{k-1} \sim \mathcal{N}(\mathbf{0}, I)$
2. Use *leapfrog integration* to compute $(\theta^*, \nu^*) = \text{leapfrog}_{\eta,L}(\theta_{k-1}, \nu_{k-1})$
3. Compute *acceptance probability* $a_k = \min(1, \exp[-H(\theta^*, \nu^*) + H(\theta_{k-1}, \nu_{k-1})])$
4. *Set* $\theta_k = \theta^*$ with probability a_k , otherwise $\theta_k = \theta_{k-1}$

- Recall $p_t(\theta, \nu) = \frac{1}{Z} e^{-H(\theta, \nu)}$ and the acceptance probability a_k in the Metropolis-algorithm:

$$a_k = \min\left(1, \frac{p_t(\theta^*, \nu^*)}{p_t(\theta_{k-1}, \nu_{k-1})}\right) = \min\left(1, \frac{\frac{1}{Z} e^{-H(\theta^*, \nu^*)}}{\frac{1}{Z} e^{-H(\theta_{k-1}, \nu_{k-1})}}\right) = \min\left(1, e^{-H(\theta^*, \nu^*) + H(\theta_{k-1}, \nu_{k-1})}\right)$$

- Note that if step-size η is sufficiently small such that $H(\theta^*, \nu^*) \approx H(\theta_{k-1}, \nu_{k-1})$, the acceptance probability will be 100%!

Tuning HMC

■ *Hyperparameters* of HMC

- Step size $\eta > 0$ for leap frog integrator
- Number of leapfrog steps L

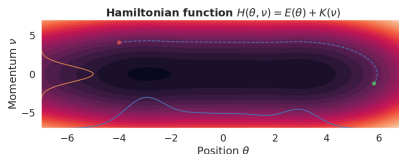
- The step-size $\eta_t > 0$ can automatically be adapted by monitoring the acceptance probability a_k . Let $\delta \in [0, 1]$ be the *desired acceptance probability*, then

$$\eta_{k+1} = \eta_k - \beta_t (\delta - a_k), \quad \beta_t = k^{-\gamma}, \quad \gamma \in (0.5, 1]$$

- Hoffman and Gelman (2014) also proposed a more advanced *dual averaging algortihm* that accounts for the fact that the acceptance probabilities behave differently in the warm up-phase vs. the sampling phase

- How about the number of leapfrog steps L ?

- The No U-turn sampler (NUTS) solves this by simulating until you encounter a "u-turn". The implementation is a bit more involved to ensure *reversibility*.



- A simpler option is either to *fix* $L \in \{10, \dots, 100\}$ or *randomize* L every iteration.

Advanced sampling methods: The MALA and ULA algorithms

The MALA algorithm

- HMC has a little brother: MALA. Recall the leapfrog equations

$$\boldsymbol{\nu}_{k+1/2} = \boldsymbol{\nu}_k - \frac{\eta}{2} \nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta}_k) \quad (\text{Half-step for } \boldsymbol{\nu})$$

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta \boldsymbol{\nu}_{k+1/2} \quad (\text{Full-step for } \boldsymbol{\theta})$$

$$\boldsymbol{\nu}_{k+1} = \boldsymbol{\nu}_{k+1/2} - \frac{\eta}{2} \nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta}_{k+1}) \quad (\text{Half-step for } \boldsymbol{\nu})$$

- Let's do a single leapfrog step, i.e. $L = 1$:

$$\begin{aligned} \boldsymbol{\theta}_{k+1}^{\text{MALA}} &= \boldsymbol{\theta}_k + \eta \boldsymbol{\nu}_{k+1/2} \\ &= \boldsymbol{\theta}_k + \eta \left(\boldsymbol{\nu}_k - \frac{\eta}{2} \nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta}_k) \right) \\ &= \boldsymbol{\theta}_k - \frac{\eta^2}{2} \nabla_{\boldsymbol{\theta}} E(\boldsymbol{\theta}_k) + \eta \boldsymbol{\nu}_k \\ &= \boldsymbol{\theta}_k + \frac{\eta^2}{2} \nabla_{\boldsymbol{\theta}} \log p_t(\boldsymbol{\theta}_k) + \eta \boldsymbol{\nu}_k, \quad \boldsymbol{\nu}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \end{aligned}$$

- The *Metropolis-Adjusted Langevin Algorithm* (MALA) is equivalent to

$$q_{\text{MALA}}(\boldsymbol{\theta}^* | \boldsymbol{\theta}_k) = \mathcal{N}(\boldsymbol{\theta}^* | \boldsymbol{\theta}_k + \frac{\eta^2}{2} \nabla_{\boldsymbol{\theta}} \log p_t(\boldsymbol{\theta}_k), \eta^2 \mathbf{I})$$

- Like MRW, but uses the gradient to bias the proposals towards high density regions

MALA and ULA

- The proposal distribution for *Metropolis-Adjusted Langevin Algorithm* (MALA):

$$q_{\text{MALA}}(\theta^* | \theta_k) = \mathcal{N}(\theta^* | \theta_k + \frac{\eta^2}{2} \nabla_{\theta} \log p_t(\theta_k), \eta^2 I)$$

- Even though HMC and MALA can be much more efficient than MRW, they will likely never win a prize for scalability
- In MALA, the gradient can be estimated using mini-batching, but the acceptance probability still requires a full pass through the dataset
- The *Unadjusted Langevin Algorithm* ULA uses the same proposal distribution as MALA, but simply *skips the Metropolis accept/reject* steps
- ULA is therefore faster, but it is also *biased* in the sense that it will not converge to the true target distribution

Overview

Algorithm	Proposal	MH Step?	Properties
MRW	$\theta^* = \theta_k + \eta \epsilon$ $\epsilon \sim \mathcal{N}(0, I)$	Yes	Random walk. Can suffer from slow exploration.
ULA	$\theta_{k+1} = \theta_k - \frac{\eta^2}{2} \nabla E(\theta_k) + \eta \epsilon$ $\epsilon \sim \mathcal{N}(0, I)$	No	Unadjusted. Follows the gradient for directed exploration. Always accepts, resulting in a <i>biased</i> stationary distribution.
MALA	$\theta^* = \theta_k - \frac{\eta^2}{2} \nabla E(\theta_k) + \eta \epsilon$ $\epsilon \sim \mathcal{N}(0, I)$	Yes	Adjusted with MH correction. Eliminates ULA's bias. Equivalent to HMC with $L = 1$. Scales better than MRW.
HMC	$(\theta^*, \nu^*) = \text{Leapfrog}_{\eta, L}(\theta_k, \nu_k)$ $\nu_k \sim \mathcal{N}(0, I)$	Yes	Simulates Hamiltonian dynamics. Suppresses random walks, allowing for distant proposals with high acceptance rates.

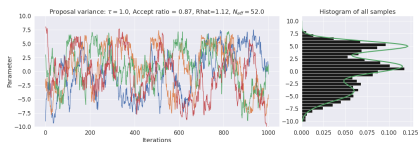
Convergence diagnostics

Are we there yet?

- MCMC theory states that samplers converge to the true target distribution as $S \rightarrow \infty$.
- *Intuitive heuristic for assessing stationarity*: Run *multiple* chains from *different initial conditions*. After K iterations, we compare the distributions for each chain. If they are different, the chains have not yet reached the stationary distribution.
- Let B denote the *between-chain variance* and let W denote the *within-chain variance*, then $\hat{R}$ -statistic (or the *potential scale reduction factor*) for chains of length N is defined by

$$\hat{R}^2 = \frac{S - 1}{S} + \frac{1}{S} \frac{B}{W}$$

- If $B = W$, then $\hat{R} = 1$. If $B > W$, then $\hat{R} > 1$. In practice, we say that the *chains have mixed* if $\hat{R} < 1.1$



For more details and motivation, see p. 284 in Bayesian Data Analysis (<http://www.stat.columbia.edu/~gelman/book/BDA3.pdf>)

How accurate is MCMC? Quantifying the error

- Recall for i.i.d. samples, the error of the MC estimator decreases with rate $1/\sqrt{S}$:

$$\mathbb{V}[\hat{f}] = \frac{1}{S} \mathbb{V}[f(w)]$$

- We can *estimate the variance* based on the samples and use this to *quantify the Monte Carlo error*. Let $\widehat{\text{sd}}(f(w))$ be the standard deviation of the MCMC samples,

$$\text{MCSE} = \frac{1}{\sqrt{S}} \widehat{\text{sd}}(f(w))$$

- MCMC methods produces *highly correlated* samples. The *autocorrelation function* ρ_t measures the correlation between two samples w^i and w^{i+t}

$$\rho_t = \frac{1}{\sigma^2} \int (w^i - \mu)(w^{i+t} - \mu) p(w) dw$$

- The *effective sample size (ESS)* S_{eff} takes this correlation into account

$$S_{\text{eff}} = \frac{S}{\sum_{t=-\infty}^{\infty} \rho_t} = \frac{S}{1 + 2 \sum_{t=1}^{\infty} \rho_t}$$

- and then

$$\text{MCSE} = \frac{1}{\sqrt{S_{\text{eff}}}} \widehat{\text{sd}}(f(w))$$

Example: MCMC diagnostics

- The $\hat{R}$ – statistic

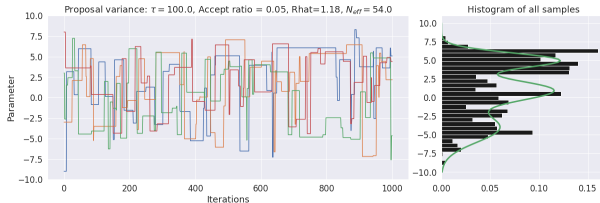
$$\hat{R}^2 = \frac{N-1}{N} + \frac{1}{N} \frac{B}{W}$$

- The effective sample size

$$S_{\text{eff}} = \frac{S}{1 + 2 \sum_{t=1}^{\infty} \rho_t}$$

- The Monte Carlo
Standard Error

$$\text{MCSE} = \frac{1}{\sqrt{S_{\text{eff}}}} \hat{\text{sd}}(f(w))$$



The detailed balance condition I

- If a chain satisfies the *detailed balance condition*, then p^* is its stationary distribution

$$T(\theta'|\theta)p^*(\theta) = T(\theta|\theta')p^*(\theta')$$

- Integrating both sides wrt. θ' yields the stationary distribution p^*

$$\int T(\theta'|\theta)p^*(\theta)d\theta' = p^*(\theta) = \int T(\theta|\theta')p^*(\theta')d\theta'$$

- Does the Metropolis-Hastings satisfy this condition? Let's check
- Recall the acceptance probability for Metropolis-Hastings

$$A(\theta^*|\theta^k) = \min \left[1, \frac{p(\theta^*)q(\theta^k|\theta^*)}{p(\theta^k)q(\theta^*|\theta^k)} \right]$$

- The transition kernel for Metropolis-Hasting

$$T(\theta'|\theta) = \begin{cases} q(\theta'|\theta)A(\theta'|\theta) & \text{if } \theta' \neq \theta \\ q(\theta|\theta)A(\theta|\theta) + \int q(\theta''|\theta) [1 - A(\theta''|\theta)] d\theta'' & \text{if } \theta' = \theta \end{cases}$$

The detailed balance condition II

- The acceptance probability

$$A(\theta^*, \theta^k) = \min \left[1, \frac{p(\theta^*)q(\theta^k|\theta^*)}{p(\theta^k)q(\theta^*|\theta^k)} \right]$$

- The transition kernel

$$T(\theta'|\theta) = \begin{cases} q(\theta'|\theta)A(\theta'|\theta) & \text{if } \theta' \neq \theta \\ q(\theta|\theta)A(\theta|\theta) + \int q(\theta''|\theta) [1 - A(\theta''|\theta)] d\theta'' & \text{if } \theta' = \theta \end{cases}$$

- If $p(\theta^k)q(\theta^*|\theta^k) > p(\theta^*)q(\theta^k|\theta^*)$, then $A(\theta^*|\theta^k) < 1$ and $A(\theta^k|\theta^*) = 1$

- To jump from θ^k to θ^* , we first need to propose it and then accept it

$$T(\theta^*|\theta^k) = q(\theta^*|\theta^k)A(\theta^*|\theta^k) = q(\theta^*|\theta^k) \frac{p(\theta^*)q(\theta^k|\theta^*)}{p(\theta^k)q(\theta^*|\theta^k)} = \frac{p(\theta^*)q(\theta^k|\theta^*)}{p(\theta^k)}$$

- It follows

$$p(\theta^k)T(\theta^*|\theta) = p(\theta^*)q(\theta^k|\theta^*)$$

The detailed balance condition III

- We just arrived at

$$p(\theta^k) T(\theta^* | \theta^k) = p(\theta^*) q(\theta^k | \theta^*)$$

- What about the opposite direction?

$$T(\theta^k | \theta^*) = q(\theta^k | \theta^*) A(\theta^k | \theta^*) = q(\theta^k | \theta^*)$$

- Combining the two and we are done

$$p(\theta^k) T(\theta^* | \theta^k) = p(\theta^*) T(\theta^k | \theta^*)$$